KU LEUVEN

FACULTEIT INDUSTRIËLE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Labo 3

Labo Geavanceerde Computerarchitectuur

Kjell NAESSENS
Beau FORREZ

Labo verantwoordelijke: T. Feys

Academiejaar 2025-2026

1 Context

In deze opdracht voeren we bewerkingen uit op een afbeelding. Eerst voeren we een inversie uit en daarna een Gaussian blur. Hierbij bekijken we de impact van het aantal threads op de uitvoeringstijd. We bekijken dus de impact van full en partial warps.

2 Methodologie

2.1 Afbeelding inverteren

Deze implementatie inverteert een afbeelding op de GPU. Iedere pixel wordt apart omgerekend naar het inverse via de formule $x = 255 - x$ waarbij $x \in (0, 255)$. We maken gebruik van striding, hierbij bekijken we het effect van verschillende block-groottes.

De grootte van een stride wordt berekend door 'int stride = blockDim.x * gridDim.x'. De kernel voert verschillende transformaties uit en neemt telkens een offset van 'stride' om het volgende element te bepalen.

De code in listing 1 toont onze uitvoering van deze opdracht.

2.2 Gaussian blur

Nu wordt de kernel aangepast zodat de rode component van de afbeelding wordt berekend als een Gaussian blur. De gebruikte formule voor pixel $r_i = 0.1r_{i-2} + 0.25r_{i-1} + 0.5r_i + 0.25r_{i+1} + 0.1r_{i+2}$. De groene en blauwe componenten worden nog steeds geïnverteerd zoals voorheen.

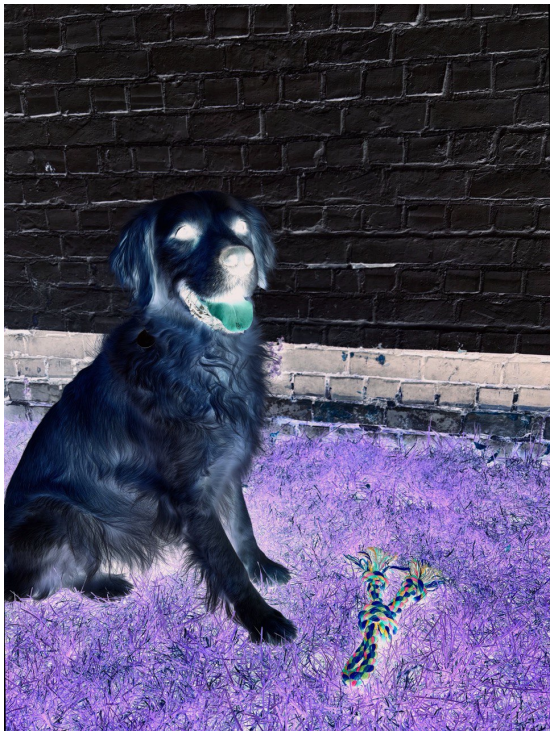
We evalueren hier ook de impact van het herformatteren van de input-data. De R-, G- en B-componenten worden gegroepeerd zodat de GPU sequentieel geheugen kan lezen (wat efficiënter is). Op deze manier voorkomen we ook thread divergence aangezien dezelfde bewerking telkens wordt uitgevoerd op opeenvolgende pixels.

De code in listing 2 toont de implementatie van de blur-kernel. Listing 3 toont de code waarbij de input-data geformatteerd wordt.

3 Observaties en resultaten

3.1 Afbeelding inverteren

Figuur 1 toont het resultaat van de inversie-kernel.



Figuur 1: Geïnverteerde afbeelding



Figuur 2: Gaussian blur op R-component

We voerden de inversie uit met toenemende hoeveelheden threads. Hierdoor moeten er telkens minder blocks worden uitgevoerd (minder warps) omdat een enkele thread meer elementen verwerkt. Figuur 3 toont de gemeten uitvoertijden voor verschillende hoeveelheden threads.

Zoals ook verwacht wordt, is er een zeer groot verschil tussen 16 en 32 threads per block. Bij 16 is er namelijk een partial warp en doet de helft van de threads niets. Wanneer we hoger gaan dan 128 threads zien we geen verbetering, dit komt waarschijnlijk door de extra overhead.

```
student@B230-PC07:/media/student/beaufor/CompArch/L3$ nvcc main.cu -o test
student@B230-PC07:/media/student/beaufor/CompArch/L3$ ./test
Threads/block   Blocks  Time (ms)       Warp type
-----------------------------------------------
16              230400  13.8337         Partial warp
32              115200  1.53245         Full warp
64              57600   0.574016                Full warp
128             28800   0.54896         Full warp
256             14400   0.561024                Full warp
512             7200    0.620928                Full warp
1024            3600    0.759808                Full warp

Output saved as output_inverted.ppm
```

Figuur 3: Impact van aantal threads (striding)

3.2 Gaussian blur

Figuur 2 toont het resultaat van de Gaussian blur op de R-component.

We zien een afname in uitvoeringstijd van 30% wanneer we de pixel-waarden formatteren naar een planaire array (zie figuur 4).

```
student@B230-PC07:/media/student/beaufor/CompArch/L3$ nvcc main3.cu -o test
student@B230-PC07:/media/student/beaufor/CompArch/L3$ ./test

Execution times:
  With divergence (interleaved RGB): 0.77088 ms
  Without divergence (planar RGB):   0.521856 ms
```

Figuur 4: Impact van divergence

4 Besluit

We stellen vast dat het veel efficiënter is om blocks een aantal threads te geven dat een veelvoud is van 32. Zo werken we met full warps, waardoor geen threads idle zijn.

Ook het belang van threads met dezelfde complexiteit is duidelijk. Wanneer deze complexiteit overal gelijk is, is er geen thread divergence, wat idle threads voorkomt. We kunnen deze divergence bij bewerkingen op afbeeldingen voorkomen door de pixels om te zetten naar een planaire sequentie.

5 Appendix

5.1 Part 1: inversie

```
1
2  #include <iostream>
3  #include <cuda_runtime.h>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <cstdio>
7
8  // ================================================================
9  // CUDA IMAGE INVERSION + PERFORMANCE ASSESSMENT
10 // ================================================================
11 // Doel:
12 //   Inverteer een .ppm-afbeelding via de GPU en analyseer hoe het
13 //   aantal threads per block (volledige of gedeeltelijke warps)
14 //   de rekentijd beinvloedt.
15 //
16 // Concept:
17 //   - Een warp = 32 threads die parallel worden uitgevoerd.
18 //   - Volledige warps (veelvoud van 32) zijn efficient.
19 //   - Gedeeltelijke warps (niet veelvoud van 32) zorgen voor
20 //     onderbenutting van GPU-resources.
21 // ================================================================
22
23 // ---- CONSTANTEN ----
24 #define M 960       // image width
25 #define N 1280      // image height
26 #define C 3         // RGB
27 #define OFFSET 16   // PPM header offset
28
29 // ---- CUDA ERROR CHECK ----
30 inline void cudaCheck(cudaError_t err) {
31     if (err != cudaSuccess) {
32         std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
33         exit(EXIT_FAILURE);
34     }
35 }
36
37 // ---- FUNCTIE: lees .ppm bestand ----
38 uint8_t* get_image_array() {
39     FILE *imageFile = fopen("anna.ppm", "rb");
40     if (imageFile == NULL) {
41         perror("ERROR: Cannot open input file (anna.ppm)");
42         exit(EXIT_FAILURE);
43     }
44
45     // sla header over
46     uint8_t header[OFFSET];
```

```
47      fread(header, 1, OFFSET, imageFile);
48
49      // lees beelddata
50      uint8_t* image_array = (uint8_t*)malloc(M * N * C);
51      if (image_array == NULL) {
52          std::cerr << "Memory allocation failed\n";
53          exit(EXIT_FAILURE);
54      }
55
56      fread(image_array, 1, M * N * C, imageFile);
57      fclose(imageFile);
58      return image_array;
59  }
60
61  // ---- FUNCTIE: schrijf .ppm bestand ----
62  void save_image_array(uint8_t* image_array, const char* filename) {
63      FILE *imageFile = fopen(filename, "wb");
64      if (imageFile == NULL) {
65          perror("ERROR: Cannot open output file");
66          exit(EXIT_FAILURE);
67      }
68
69      // schrijf header
70      fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
71      fwrite(image_array, 1, M * N * C, imageFile);
72      fclose(imageFile);
73  }
74
75  // ---- CUDA KERNEL ----
76  // Inverteert elk RGB-component (255 - waarde)
77  __global__ void invert_kernel_strided(const uint8_t* in, uint8_t* out, int
        total) {
78      int idx = blockIdx.x * blockDim.x + threadIdx.x;
79      int stride = blockDim.x * gridDim.x;
80
81      for (int i = idx; i < total; i += stride) {
82          out[i] = 255 - in[i];
83      }
84  }
85
86
87  // ---- MAIN ----
88  int main() {
89      const int totalPixels = M * N * C;
90      const size_t totalBytes = totalPixels * sizeof(uint8_t);
91
92      // lees invoerbeeld
93      uint8_t* h_in = get_image_array();
94      uint8_t* h_out = (uint8_t*)malloc(totalBytes);
95
96      // device geheugen
```

```
97      uint8_t *d_in, *d_out;
98      cudaCheck(cudaMalloc(&d_in, totalBytes));
99      cudaCheck(cudaMalloc(&d_out, totalBytes));
100     cudaCheck(cudaMemcpy(d_in, h_in, totalBytes, cudaMemcpyHostToDevice));
101
102     // events voor timing
103     cudaEvent_t start, stop;
104     cudaCheck(cudaEventCreate(&start));
105     cudaCheck(cudaEventCreate(&stop));
106
107     // thread-configuraties om te testen
108     int blockSizes[] = {16, 32, 64, 128, 256, 512, 1024};
109     int numConfigs = sizeof(blockSizes) / sizeof(blockSizes[0]);
110
111     std::cout << "Threads/block\tBlocks\tTime (ms)\tWarp type\n";
112     std::cout << "-----------------------------------------------\n";
113
114     // test verschillende aantallen threads per block
115     for (int i = 0; i < numConfigs; i++) {
116         int threads = blockSizes[i];
117         int blocks = (totalPixels + threads - 1) / threads;
118
119         cudaEventRecord(start);
120         invert_kernel_strided<<<blocks, threads>>>(d_in, d_out, totalPixels);
121         cudaEventRecord(stop);
122         cudaCheck(cudaEventSynchronize(stop));
123
124         float ms = 0;
125         cudaCheck(cudaEventElapsedTime(&ms, start, stop));
126
127         // bepaal of het een volledige warp is (veelvoud van 32)
128         bool fullWarp = (threads % 32 == 0);
129
130         std::cout << threads << "\t\t"
131                   << blocks << "\t"
132                   << ms << "\t\t"
133                   << (fullWarp ? "Full warp" : "Partial warp") << "\n";
134     }
135
136     // kopieer resultaat terug
137     cudaCheck(cudaMemcpy(h_out, d_out, totalBytes, cudaMemcpyDeviceToHost));
138
139     // opslaan
140     save_image_array(h_out, "output_inverted.ppm");
141
142     // cleanup
143     cudaFree(d_in);
144     cudaFree(d_out);
145     free(h_in);
146     free(h_out);
147     cudaEventDestroy(start);
```

```
148     cudaEventDestroy(stop);
149
150     std::cout << "\nOutput saved as output_inverted.ppm\n";
151     std::cout << "Analysis complete: full vs. partial warp performance assessed
            .\n";
152     return 0;
153 }
```

Listing 1: inversie

5.2 Part 2: Gaussian blur

```
1
2  #include <iostream>
3  #include <cuda_runtime.h>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <cstdio>
7
8  // ---- CONSTANTEN ----
9  #define M 960       // image width
10 #define N 1280      // image height
11 #define C 3         // RGB
12 #define OFFSET 16   // PPM header offset
13
14 // ---- CUDA ERROR CHECK ----
15 inline void cudaCheck(cudaError_t err) {
16     if (err != cudaSuccess) {
17         std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
18         exit(EXIT_FAILURE);
19     }
20 }
21
22 // ---- FUNCTIE: lees .ppm bestand ----
23 uint8_t* get_image_array() {
24     FILE *imageFile = fopen("anna.ppm", "rb");
25     if (!imageFile) { perror("ERROR opening input file"); exit(EXIT_FAILURE); }
26
27     uint8_t header[OFFSET];
28     fread(header, 1, OFFSET, imageFile);
29
30     uint8_t* image_array = (uint8_t*)malloc(M * N * C);
31     fread(image_array, 1, M * N * C, imageFile);
32     fclose(imageFile);
33
34     return image_array;
35 }
36
37 // ---- FUNCTIE: schrijf .ppm bestand ----
38 void save_image_array(uint8_t* image_array, const char* filename) {
```

```
    FILE *imageFile = fopen(filename, "wb");
    if (!imageFile) { perror("ERROR opening output file"); exit(EXIT_FAILURE);
        }

    fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

// ---- CUDA KERNEL: Gaussian blur ----
__global__ void divergence_kernel(const uint8_t* in, uint8_t* out, int width,
    int height) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int totalPixels = width * height * C;
    if(idx < totalPixels -2 && idx > 2 && idx %3 == 0){
        out[idx] = 0.1 *in[idx -2*3]  +  0.25* in[idx -1*3] + 0.5*in[idx] +
            0.25*in[idx +1 *3] +0.1* in[idx +2*3];
    }
    else if (idx <= totalPixels && idx >=0){
        out[idx] =255 -  in[idx];
    }
}

// ---- MAIN ----
int main() {
    const int totalPixels = M * N * C;
    const size_t totalBytes = totalPixels * sizeof(uint8_t);

    uint8_t* h_in = get_image_array();
    uint8_t* h_out = (uint8_t*)malloc(totalBytes);

    uint8_t *d_in, *d_out;
    cudaCheck(cudaMalloc(&d_in, totalBytes));
    cudaCheck(cudaMalloc(&d_out, totalBytes));
    cudaCheck(cudaMemcpy(d_in, h_in, totalBytes, cudaMemcpyHostToDevice));

    cudaEvent_t start, stop;
    cudaCheck(cudaEventCreate(&start));
    cudaCheck(cudaEventCreate(&stop));

    int threads = 256;
    int blocks = (totalPixels + threads - 1) / threads;

    cudaEventRecord(start);
    divergence_kernel<<<blocks, threads>>>(d_in, d_out, N, M);
    cudaEventRecord(stop);
    cudaCheck(cudaEventSynchronize(stop));

    float ms = 0;
    cudaCheck(cudaEventElapsedTime(&ms, start, stop));
    std::cout << "Thread divergence kernel time: " << ms << " ms\n";
```

```
87
88      cudaCheck(cudaMemcpy(h_out, d_out, totalBytes, cudaMemcpyDeviceToHost));
89      save_image_array(h_out, "output_divergence.ppm");
90
91      cudaFree(d_in);
92      cudaFree(d_out);
93      free(h_in);
94      free(h_out);
95      cudaEventDestroy(start);
96      cudaEventDestroy(stop);
97
98      std::cout << "Output saved as output_divergence.ppm\n";
99      return 0;
100 }
```

Listing 2: Rode bewerking

5.3 Part 3: R...RG...GB...B

```
1
2  #include <iostream>
3  #include <cuda_runtime.h>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <cstdio>
7
8  // ---- CONSTANTEN ----
9  #define M 960       // image width
10 #define N 1280      // image height
11 #define C 3         // RGB
12 #define OFFSET 16   // PPM header offset
13
14 // ---- CUDA ERROR CHECK ----
15 inline void cudaCheck(cudaError_t err) {
16     if (err != cudaSuccess) {
17         std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
18         exit(EXIT_FAILURE);
19     }
20 }
21
22 // ---- LEES .ppm ----
23 uint8_t* get_image_array() {
24     FILE *imageFile = fopen("anna.ppm", "rb");
25     if (!imageFile) {
26         perror("ERROR: Cannot open input file (anna.ppm)");
27         exit(EXIT_FAILURE);
28     }
29
30     uint8_t header[OFFSET];
31     fread(header, 1, OFFSET, imageFile);
```

```
32
33      uint8_t* image_array = (uint8_t*)malloc(M * N * C);
34      if (!image_array) {
35          std::cerr << "Memory allocation failed\n";
36          exit(EXIT_FAILURE);
37      }
38
39      fread(image_array, 1, M * N * C, imageFile);
40      fclose(imageFile);
41      return image_array;
42  }
43
44  // ---- SCHRIJF .ppm ----
45  void save_image_array(uint8_t* image_array, const char* filename) {
46      FILE *imageFile = fopen(filename, "wb");
47      if (!imageFile) {
48          perror("ERROR: Cannot open output file");
49          exit(EXIT_FAILURE);
50      }
51
52      fprintf(imageFile, "P6\n%d %d\n255\n", M, N);
53      fwrite(image_array, 1, M * N * C, imageFile);
54      fclose(imageFile);
55  }
56
57  // ---- KERNEL 1: INTERLEAVED RGB (met divergence) ----
58  __global__ void divergence_kernel(const uint8_t* in, uint8_t* out, int width,
        int height) {
59      int idx = blockIdx.x * blockDim.x + threadIdx.x;
60      int total = width * height * C;
61      if (idx >= total) return;
62
63      int pixelIndex = idx / C;
64      int color = idx % C;
65
66      if (color == 0) {  // R: Gaussian blur
67          int x = pixelIndex % width;
68          float sum = 0.0f;
69          for (int k = -2; k <= 2; k++) {
70              int nx = x + k;
71              if (nx >= 0 && nx < width) {
72                  int neighborIdx = (pixelIndex + k) * C;
73                  float weight;
74                  switch (k) {
75                      case -2:
76                      case 2:  weight = 0.1f; break;
77                      case -1:
78                      case 1:  weight = 0.25f; break;
79                      default: weight = 0.5f;  break;
80                  }
81                  sum += weight * in[neighborIdx];
```

```
            }
        }
        out[idx] = static_cast<uint8_t>(sum);
    } else {  // G and B: invert
        out[idx] = 255 - in[idx];
    }
}

// ---- KERNEL 2: PLANAR RR...GG...BB... (zonder divergence) ----
__global__ void planar_kernel(const uint8_t* in, uint8_t* out, int width, int
    height) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int pixels = width * height;

    // Gaussian blur alleen op R-plane, index veilig voor +/-2
    if (idx >= 2 && idx < pixels - 2) {
        out[idx] = 0.1f * in[idx - 2] + 0.25f * in[idx - 1] + 0.5f * in[idx]
                 + 0.25f * in[idx + 1] + 0.1f * in[idx + 2];
    }
    else if (idx < pixels) {
        // randen of kleine matrices: kopieer origineel
        out[idx] = in[idx];
    }
    else if (idx < 2 * pixels) {
        // G-plane invert
        out[idx] = 255 - in[idx];
    }
    else {
        // B-plane invert
        out[idx] = 255 - in[idx];
    }
}

// ---- HERSCHIK NAAR PLANAR ----
uint8_t* convert_to_planar(const uint8_t* interleaved) {
    int pixels = M * N;
    uint8_t* planar = (uint8_t*)malloc(pixels * C);

    for (int i = 0; i < pixels; i++) {
        planar[i] = interleaved[i * C];         // R
        planar[i + pixels] = interleaved[i * C + 1]; // G
        planar[i + 2 * pixels] = interleaved[i * C + 2]; // B
    }
    return planar;
}

// ---- TERUG NAAR INTERLEAVED ----
uint8_t* convert_to_interleaved(const uint8_t* planar) {
    int pixels = M * N;
    uint8_t* interleaved = (uint8_t*)malloc(pixels * C);

```

```
    for (int i = 0; i < pixels; i++) {
        interleaved[i * C]     = planar[i];
        interleaved[i * C + 1] = planar[i + pixels];
        interleaved[i * C + 2] = planar[i + 2 * pixels];
    }
    return interleaved;
}

// ---- MAIN ----
int main() {
    const int total = M * N * C;
    const size_t totalBytes = total * sizeof(uint8_t);

    // lees invoerbeeld
    uint8_t* h_in = get_image_array();

    // --- 1. Divergence (interleaved RGB) ---
    uint8_t *d_in, *d_out;
    cudaCheck(cudaMalloc(&d_in, totalBytes));
    cudaCheck(cudaMalloc(&d_out, totalBytes));
    cudaCheck(cudaMemcpy(d_in, h_in, totalBytes, cudaMemcpyHostToDevice));

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    int threads = 256;
    int blocks = (total + threads - 1) / threads;

    cudaEventRecord(start);
    divergence_kernel<<<blocks, threads>>>(d_in, d_out, N, M);
    cudaEventRecord(stop);
    cudaCheck(cudaEventSynchronize(stop));
    float t_div = 0;
    cudaEventElapsedTime(&t_div, start, stop);

    uint8_t* h_div = (uint8_t*)malloc(totalBytes);
    cudaMemcpy(h_div, d_out, totalBytes, cudaMemcpyDeviceToHost);
    save_image_array(h_div, "output_divergence.ppm");

    // --- 2. Planar (zonder divergence) ---
    uint8_t* h_planar_in = convert_to_planar(h_in);
    uint8_t* h_planar_out = (uint8_t*)malloc(totalBytes);

    cudaMemcpy(d_in, h_planar_in, totalBytes, cudaMemcpyHostToDevice);

    blocks = (total + threads - 1) / threads;
    cudaEventRecord(start);
    planar_kernel<<<blocks, threads>>>(d_in, d_out, N, M);
    cudaEventRecord(stop);
    cudaCheck(cudaEventSynchronize(stop));
```

```
    float t_planar = 0;
    cudaEventElapsedTime(&t_planar, start, stop);

    cudaMemcpy(h_planar_out, d_out, totalBytes, cudaMemcpyDeviceToHost);

    uint8_t* h_planar_interleaved = convert_to_interleaved(h_planar_out);
    save_image_array(h_planar_interleaved, "output_planar.ppm");

    std::cout << "\nExecution times:\n";
    std::cout << "  With divergence (interleaved RGB): " << t_div << " ms\n";
    std::cout << "  Without divergence (planar RGB):   " << t_planar << " ms\n"
        ;

    // cleanup
    cudaFree(d_in);
    cudaFree(d_out);
    free(h_in);
    free(h_div);
    free(h_planar_in);
    free(h_planar_out);
    free(h_planar_interleaved);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return 0;
}
```

Listing 3: Planar RGB